推荐经典模型和 Loss 推演 Part 1

从双塔到 FM、DCN、DLRM 与 DIN

Bo Hou

2026-10-30

日期：2026-10-30
本文是一份围绕推荐系统建模展开的推演笔记，来自一天的连续讨论。重点不是背模型名称，而是追问：数据怎么构造，模型希望学到什么，结构为什么这样设计，以及参数如何被优化。
文中的“我问 / 讨论 / 修正”是对讨论的整理与重述，并非逐字聊天记录。已修正讨论中出现的符号混用、过强结论和不准确表述。本文不声称这些推导具有论文原创性。

目录

1. 一条统一主线：数据、建模、优化与验证
2. 推荐问题从哪里开始：热门偏置与系统诊断
3. 双塔召回：输入、结构、训练和上线边界
4. 用户历史编码：从加权平均走到 self-attention
5. 时间截断、训练窗口与缓存更新
6. 从 concat + MLP 推到显式交互
7. FM：把巨大的类别交叉系数表变成 embedding 内积
8. DeepFM 与 Wide & Deep：两条不同的组合路线
9. DCN v1：亲手展开一次显式多项式交叉
10. DCNv2：从整体缩放走到逐维交叉
11. 低秩分解与 LoRA：压缩的究竟是什么
12. DLRM：保留交互分数，再学习非线性组合
13. DIN：让候选决定哪些历史行为更重要
14. 今天涉及的 Loss：BCE、in-batch CE、logQ 与多正例
15. 附录：显著性、AUROC/NDCG 与特征重要性
16. Part 2 从哪里继续

1. 一条统一主线：数据、建模、优化与验证

讨论到最后，我觉得很多 AI/ML 问题都能沿着三个问题拆开：

1. **数据构造是什么？** 一条样本代表什么，特征是什么，label 是什么，样本来自什么分布？
2. **怎么建模，究竟学什么？** 希望模型捕捉哪些关系，为什么这些关系适合用这个结构表达？
3. **优化怎么走？** loss 如何表达目标，梯度传到哪里，参数如何求解，计算是否可承受？

再补一个验证闭环：**怎么证明它学到的东西有用？** 包括离线指标、统计不确定性、线上实验以及部署成本。

这个框架解释了为什么 FM、DeepFM、DCN、DLRM 看起来各有公式，但不是一堆彼此孤立的技巧。它们常常面对相似的输入与点击 label，也可以采用同一种 BCE；真正不同的是结构对关系的表达方式。

例如，MLP 可以学习复杂非线性，但这不意味着每一种关系都同样容易学。显式提供两两内积或乘积，相当于告诉模型：“这种交互值得优先表达。”这是一种结构归纳偏置，而不是宣布 MLP 没有相应的表达能力。

2. 推荐问题从哪里开始：热门偏置与系统诊断

2.1 先判断问题在哪，再提出模型

最初的场景是：推荐结果被热门视频主导，希望改善个性化和长尾发现，同时让 CTR 等已有业务指标至少不退化。

我的第一反应是列方案：点击历史 boost、video-to-video 传播、轻量双塔。但很快意识到，更合理的起点是：

> 先看 popularity 问题来自 retrieval、ranking，还是缺少 exploration。系统现状没有弄清楚之前，不要急着堆方案。

可以检查完整漏斗：

- 全量可推荐视频的热门程度分布。
- retrieval 候选集的分布。
- ranking 后曝光集的分布。
- 最终点击或高质量观看集的分布。

例如候选集本来就没有长尾视频，ranking 很难把不存在的内容排上来；候选集有长尾，但曝光阶段几乎消失，就更值得检查 ranking 的打分与约束。

不过，漏斗分布只能提供定位线索，不能单凭某一层熵较低就证明根因。候选资格、用户真实偏好、库存和业务规则都可能影响分布。

2.2 “多样性”不等于“降低热门偏置”

讨论中有一个重要修正：diversity 并不自动代表解决 popularity。

一组视频可以题材很丰富，但全部来自热门头部；也可以全部来自长尾，却高度同质。

更直接的做法，是按视频在实验前固定时间窗内的点击量、曝光量等定义 popularity 分层，再比较：

- head / mid / tail 的曝光占比与点击占比。
- 每个 popularity 桶的点击数量、CTR 和观看质量。
- 被发现的长尾视频数量、覆盖率。
- 相同总曝光预算下，流量分配是否变化。

分桶边界和 popularity 口径应在实验前固定，并对两个实验组一致。 否则实验自身改变点击量后，视频会跨桶，使比较口径也跟着变化。

原始讨论里说“曲线往左靠”，更准确的表达是：如果横轴为事先定义的视频 popularity、越左越长尾，那么长尾部分的点击质量或占比提高。仅看直方图位置不够，还要明确画的是视频数量、曝光、点击还是归一化占比。

2.3 逐级 baseline，而不是上来就训练大模型

方案	直觉	需要检查的边界
Control	当前产品	保留真实基准
Base 1	用用户自己的点击历史给视频或相关候选 boost	看过不等于还应该推荐；重复消费与新内容发现要区分
Base 2	用户→视频信号，再沿视频→视频关系传播一次	传播是否增加覆盖，还是进一步放大头部
Base 3	学习用户与视频的共享向量空间，用 ANN 召回	数据、loss、服务延迟和索引更新

若当前 ranking 已有

$$s(u, v) = w_1 p(\text{click} \mid u, v) + w_2 \text{popularity}(v),$$

可以先消融或调整 popularity 项，检查是不是显式热门信号压过了个性化。

但调小 popularity 权重本身不等于 exploration 算法：它可以作为诊断实验或流量重分配实验，正式探索还涉及不确定性、随机策略和长期反馈。

3. 双塔召回：输入、结构、训练和上线边界

3.1 我最初想搭的 baseline

用户侧可以包括：user ID、已有 user embedding、年龄、地区、用户群组，以及历史行为编码。

视频侧可以包括：video ID、视频团队提供的内容 embedding、文本 embedding、音频 embedding、类别、创建时间，以及在预测时已知的统计特征。

两边分别 concat，再各过一个小 MLP，投影到同一个空间：

$$q_u = f_\theta(\text{user features}), \qquad k_v = g_\phi(\text{video features}),$$

$$q_u, k_v \in \mathbb{R}^d, \qquad s(u, v) = q_u^\top k_v.$$

必要时可做 L2 normalization 和温度缩放。特征清单不是越长越好：可用性、信息增益、隐私要求、计算与更新成本都要考虑，名字等字段也不应默认加入。

3.2 “这不就是 CLIP 吗？”

讨论中的直觉是：两个 encoder，映射到共享空间，正例靠近、负例远离，这不就是同一类结构吗？

这个直觉在架构层面成立：都是双编码器和对比学习的思路。但 CLIP 是具体的图文预训练方法名称；推荐系统里用用户、视频及多种特征构建双编码器，称为 two-tower 更清楚。

不必为了名称把“仅已有 embedding 的双塔”和“加入更多 sparse/dense 特征的双塔”强行当成两个本质不同的模型。先做一个可扩展 baseline 即可。

3.3 一个关键的上线约束：两边必须能独立计算

我一开始希望 forward 接收 user info、video info、interaction info。问题是：interaction 中哪些属于特征，哪些只属于监督与样本元数据?

如果是当前待预测用户与候选视频的 pairwise 特征，例如二者共同计算出的复杂交互，直接混进两塔，就可能破坏独立编码。

标准 ANN 双塔要求：

- 视频向量可提前离线计算并建立索引。
- 用户向量不依赖当前正在打分的某个候选。
- 在线用统一相似度检索视频向量。

因此，当前 pair 的点击 label、样本权重、session ID、采样概率等可以用于训练，但不能因此自动成为两塔的预测输入。

用户的**过去交互**可以进入用户历史 encoder；视频在当前预测前已知的全局统计可以进入视频侧。关键是来源、时间和候选依赖，而不是看到“interaction”这个名字就一概排除。

候选相关的交互建模通常更适合 ranking，后面 DIN 就是一个例子。

4. 用户历史编码：从加权平均走到 self-attention

4.1 第一个想法：行为与时间加权

用户历史可以写成：

$$H_u(t) = \{(o_i, a_i, T_i) : T_i < t\},$$

其中 o_i 是行为对象，a_i 是点击、观看、点赞等动作，T_i 是时间。

把对象和动作等编码成 e_i，再用动作分数和时间衰减聚合：

$$h_u = \frac{\sum_i s(a_i)\,\delta(t - T_i)\,e_i}{\sum_i s(a_i)\,\delta(t - T_i)}.$$

这是合理 baseline。比如高质量观看可以比短点击权重大，近期行为可以比很久以前的行为权重大。

但过去动作是历史输入，当前预测目标的动作才是 label；不能因为动作能充当监督信号，就把历史里所有 action 都当作不能使用的 label。

4.2 来回追问：喜剧和惊悚平均后怎么办?

讨论提出的问题： 用户同时喜欢喜剧和惊悚，平均后的一个向量会不会落在两者中间，丢失多个兴趣?

我的反问： 如果其他视频都很远，两个兴趣簇仍然最近，不就还能召回?

这个反问成立。不能把“平均表示可能有瓶颈”说成“平均后必然失败”。是否失败取决于向量空间、候选分布、向量维度和检索方式。

更严格的问题是：**聚合是否丢失了下游需要区分的信息?**

假设固定编码下，两组不同历史有同一个平均值，且其他输入相同，那么任何只接收这个平均值的下游 MLP，都不能区分两组历史。下游参数再多，也无法恢复输入中已经消失的差别。

若 encoder 与推荐目标联合训练，模型可以调整行为表示，减轻这种碰撞；但固定聚合操作本身仍然有信息瓶颈。

4.3 一度走向 GMM 和 global cluster

我想到：把喜剧中心和惊悚中心分别输入，或者对全局行为 embeddings 聚类，让用户 embedding 去 gate 不同中心。

这里有几个容易混在一起的问题：

1. 聚类的数据究竟是什么：用户、视频，还是交互表示?
2. cluster 是全局共享，还是每个用户单独拟合?
3. 用户的多条历史究竟怎样分配到多个兴趣?
4. 最终输出一个向量还是多个向量?

全局共享 cluster 可以作为原型或兴趣字典，但如果输入仍是同一个已丢失信息的平均向量，后面再 gate，不能保证恢复原始历史。

每个用户单独拟合 GMM 也不是必要路线，尤其在历史短、更新频繁时，估计和服务成本都值得怀疑。

GMM 的协方差形状本身是：

类型	存储形状	含义
full	$K \times d \times d$	每个簇完整协方差
tied	$d \times d$	所有簇共享协方差
diag	$K \times d$	每簇各维方差，不建模维间协方差
spherical	K	每簇一个各向同性方差

这些可以学，但当时的主问题其实是用户历史 encoder，不是必须推到 GMM。这里应及时收敛，避免为了引出多兴趣而把路线越绕越远。

4.4 回到我真正想说的 attention

我提出：历史 embeddings 并不一定要先固定平均。原始序列过 self-attention 后，每个位置已经包含了历史关系；最后一个有效位置的输出可以作为用户状态。

这才对齐了讨论对象。之前的“给独立 embedding 加权平均”与“先经过序列 encoder”不是同一层操作。

一个简化的 causal self-attention encoder：

$$E = [e_1; \dots; e_T], \qquad Q = EW_Q,\ K = EW_K,\ V = EW_V,$$

$$\text{Attention}(E) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V.$$

M 是 causal/padding mask。还需要 position 或时间信息，**self-attention 本身不会自动知道顺序**。

多层变换后，取最后一个有效位置的 hidden state：

$$h_u(t) = \text{HistoryEncoder}(H_u(t))_{\text{last}}.$$

然后把它作为用户塔的一部分输入。

这里有两点修正：

- 最后 hidden state 是用于预测未来的历史表示，不必把它解释成“已经生成了下一动作 embedding”。不需要为得到这个 hidden state 再凭空生成一次动作。
- attention 包含加权聚合，因此不必再人为重复一遍相同的 pooling；但它也不意味着永远无需任何 pooling、时间建模或多兴趣设计。

5. 时间截断、训练窗口与缓存更新

5.1 泄漏问题比“历史早于 label”更细

基本要求是：在预测时间 t，输入中的全部信息必须已可获得。

这包括：

- 历史事件发生时间早于 t。
- 用户 embedding 的生成输入没有包含未来行为。
- 时间衰减、动作权重等依赖的统计量在 t 时已知。
- 视频累计点击等特征不能用数据集末尾的最终统计去回填过去。
- 若模拟严格的过去预测未来，特征 encoder 的训练数据与模型版本也要符合该评估协议。

如果预测定义是“用 t 前的信息预测 t 之后的行为”，就按这个协议构造；若 label 为发生在 t 的当前行为，则输入截止在该行为前。重点是定义一致，而不是机械争论符号用 $< t$ 还是 $\leq t$。

5.2 滑动窗口：100 条行为，窗口 30

设序列有 100 个行为，用前 30 条预测紧接的第 31 条，然后滑动到 2–31 预测 32，直到 70–99 预测 100，就得到 70 个样本。

每个样本都可以同时训练 history encoder 和双塔。相邻样本不必放进同一 batch，也不必按时间顺序执行优化。

我提出：“每条样本内部，feature 早于 label，不就行了吗？”

对训练样本内部是必要条件，但评估还有一个边界：随机拆分重叠窗口，可能让后期训练样本包含较早测试目标。

例如：

- 测试：行为 1–30 → 预测 31。
- 训练：行为 2–31 → 预测 32。

第二个训练样本把测试目标 31 当成历史输入。这样的评估不能直接代表严格的过去训练、未来预测。

因此若目标是时间外推评估，应先设全局时间边界划分训练、验证、测试，再构造符合协议的窗口。**训练集内部依然可以随机打乱 batch。** 测试时间点之前、线上已经发生的行为能否进入测试历史，也应按真实服务过程定义，而不是一概禁止。

5.3 重叠窗口仍有计算成本

把相邻窗口拆到不同 batch 解决的是调度问题，并没有消除重复编码。训练期间 encoder 参数持续变化，通常不能把之前参数下的 hidden state 直接当成当前可反传结果复用。

可按需要采用窗口采样、限制历史长度、一次序列前向监督多个位置等方式降低重复计算；但具体实现须和 label、负采样及 padding/mask 对齐。

5.4 联合训练，不等于必须在线联合执行

我最初认为：“训练一起，线上就一起；训练分开，线上就分开。”

修正后是：**训练依赖关系与服务执行时机是两件事。**

history encoder 和 user tower 可以联合训练。上线时，较重的历史编码可预计算、缓存，用户请求到来后读取缓存，再结合实时上下文计算较轻的 user tower。

这带来新问题：缓存的新鲜度、模型版本、输入特征版本和视频索引版本如何保持兼容。

用户历史更新到今天、视频向量还是昨天，不自动构成 feature leakage。若使用的都是线上当时已知信息，更多是新鲜度、分布漂移和表示兼容性问题。

5.5 新行为与 KV cache

在 causal encoder 下，新行为到来时可以考虑复用旧位置的 KV，新位置再计算自己的 query/key/value 和 attention。

但不是“只有最后一个 $QK^\top$ 变化”：新位置在每层都会产生新 hidden state、Q/K/V，还要执行该层其他变换。模型换版本后旧 KV 通常也不能直接复用。

如果是严格滑动窗口、旧事件被移出窗口，保留位置的 hidden states 可能已经受被移出事件影响，因此简单丢弃最早 KV 不一定等价于对新窗口从头计算。缓存策略必须和上下文定义一致。

6. 从 concat + MLP 推到显式交互

6.1 “MLP 已经很强，为什么还需要交叉？”

我的直觉是：两个 embedding concat 后过多层 MLP，容量已经很强，再加隐藏层不就行了？

这个起点没有错。引入交互结构不是因为 MLP 理论上绝对学不了，而是为了更直接、有效地表达某些乘法关系。

例如 $e_f, e_g \in \mathbb{R}^{16}$，外积得到：

$$e_f e_g^\top \in \mathbb{R}^{16\times 16}.$$

其元素是 $(e_f)_a (e_g)_b$，表示两个向量坐标之间的乘积项。

“物理意义”不应说成第 3 维必然是喜剧、第 7 维必然是年龄。embedding 坐标通常没有固定人工语义；交叉结构表达的是潜在表示之间的关系，具体含义随训练形成。

6.2 “外积后压缩成一个标量，用 256×1 线性层”

这一步自然得到：

$$s_{fg} = \text{vec}(e_f e_g^\top)^\top \text{vec}(W) = e_f^\top W e_g,$$

其中 $W \in \mathbb{R}^{16\times 16}$。它是一个双线性打分。

同一个 W 可以对所有 field pair 共享。共享是参数复用，不需要手工为每一对复制一个线性层；所有 pair 对同一参数的梯度会累积。

用代码形状看：

```
# E: [batch, num_fields, d]
# W: [d, d]
projected = E @ W
scores = projected @ E.transpose(-1, -2)
# scores[b, f, g] = E[b, f] @ W @ E[b, g]
# 选 f < g；若 W 非对称，应固定 field 顺序。
```

此时 W 只有 d^2 参数，但 pair 计算数量仍随 field 数量的平方增长。参数量和计算量要分开。

7. FM：把巨大的类别交叉系数表变成 embedding 内积

7.1 先统一最容易混淆的符号

符号	含义	示例
F	field 数量	100 个字段
C_f	第 f 个 field 的类别数	每字段 100 类
$M = \sum_f C_f$	拼接 one-hot 后总维度	10,000
d	embedding 维度	16
$D = Fd$	当前样本 embeddings concat 长度	1,600
W	embedding 坐标间双线性矩阵	16×16
C	原始类别坐标间交叉系数矩阵	10,000×10,000

讨论中最大的符号问题，就是先把 16×16 的矩阵叫 A，随后又把 10,000×10,000 的原始交叉表叫 A，容易让人误以为同一个对象突然变大。

以后应该明确区分：**类别空间的系数矩阵**和**embedding 空间的变换矩阵**。

7.2 field 和 category 到底是什么?

field 是字段，例如地域、语言、设备。category 是某个 field 的具体取值，例如 NJ、中文、iPhone。

“100 fields，每个 100 categories”表示每条样本通常在每个字段选一个类别，并不是一条样本同时含有一万个激活值。

one-hot 拼接后 $x \in \mathbb{R}^{10,000}$，但若每个 field 单选，则通常只有 100 个位置非零。

实际实现不必构造稠密 one-hot，而是保留类别 ID 并 lookup：

$$e_f = E_f[\text{category_id}_f].$$

hash 也只是将原始类别映射到有限桶 ID，再查 embedding；它不是语义上的区间分桶，而且可能有碰撞。

7.3 为什么直接记类别 pair 会大?

每一对不同 field，若各有 100 类，就有 10,000 个类别组合。

100 个 field 有：

$$\binom{100}{2} = 4,950$$

对，因此单独为每个跨 field 类别组合学习系数，需要：

$$4,950 \times 10,000 = 49,500,000$$

个二阶系数。

若用完整 10,000×10,000 矩阵存储则有一亿个位置，但包含对称重复、对角项和同 field 项，不能把一亿都说成独立有效交互参数。

输入很稀疏能减少当前样本的计算，却不能自动消除整张参数表的存储与估计难度。

7.4 FM 的低秩想法

原始二阶模型可以写成：

$$s = b + \sum_{i=1}^{M} w_i x_i + \sum_{i<j} C_{ij} x_i x_j.$$

FM 把系数改写为：

$$C_{ij} = \langle v_i, v_j \rangle, \qquad v_i \in \mathbb{R}^d.$$

于是：

$$s_{\text{FM}} = b + \sum_{i} w_i x_i + \sum_{i<j} \langle v_i, v_j \rangle x_i x_j.$$

这里 i, j 是拼接 one-hot 中的**类别位置编号**，不是 field 编号。$x_i x_j$ 说明对应两个类别是否同时激活。

若写成我们实际讨论的 lookup 形式，单选类别特征就更直观：

$$s_{\text{interaction}} = \sum_{f<g} \langle e_f, e_g \rangle.$$

当前类别被选中这件事，已经由 lookup 完成。 不必再显式乘一个 one-hot 指示值。

embedding 参数规模是 $Md = 10,000 \times 16 = 160,000$，而不必为约 4,950 万个类别组合各学一个独立系数。

7.5 “是不是把 W 强行设成单位矩阵？”

相对之前的 $e_f^\top W e_g$，FM 使用：

$$e_f^\top I e_g = e_f^\top e_g.$$

所以这个理解在结构上成立：省去显式 W，让 embedding 本身学习关系。

但不能说任意 W 都能无损吸收入同一套 embedding。一般双线性矩阵可以表达不对称或不定关系，而共享内积有自己的结构限制。FM 的收益来自参数共享与低秩约束，不是对任意完整交叉表的等价改写。

另外，“N×K 参数”的说法中，N 若表示 M 个类别位置，K 表示 embedding 维度，就对应 Md。K 不是有多少张独立 embedding 表，而是向量长度。

7.6 标准 FM 的内积之后，有没有可学习权重？

这一点反复讨论过，值得单列：

标准 FM：

$$\sum_{f<g} \langle e_f, e_g \rangle.$$

内积后直接求和，没有额外的 field-pair 权重。可学习的是 embeddings，以及独立的一阶项参数。

我提出的加权版本：

$$\sum_{f<g} a_{fg} \langle e_f, e_g \rangle.$$

其中 a_{fg} 也可学习，这是合理扩展，但不能直接当作标准 FM 的原始定义。

不要把“embedding 内积本身是类别交互的可学习系数”和“内积结果后再乘一个独立权重”混为一谈。

7.7 FM 为什么不一定需要枚举所有 pair？

利用恒等式：

$$\sum_{i<j} \langle v_i, v_j \rangle x_i x_j = \frac{1}{2} \sum_{k=1}^{d} \left[\left(\sum_i v_{ik} x_i \right)^2 - \sum_i v_{ik}^2 x_i^2 \right].$$

若有 m_{active} 个非零输入，二阶项可以在 $O(m_{\text{active}}d)$ 时间求和，而不显式构造全部 pair。

但若需要保留每一对分数，或加上任意 a_{fg}，就不能默认仍可使用这个同样的求和简化。

7.8 lookup 在网络里怎么表示?

```
import torch
from torch import nn

class LookupAndFM(nn.Module):
  def __init__(self, num_fields=100, categories_per_field=100, d=16):
    super().__init__()
    self.tables = nn.ModuleList([
      nn.Embedding(categories_per_field, d)
      for _ in range(num_fields)
    ])

  def forward(self, category_ids):
    # category_ids: [B, F]，每一列是一个 field 的类别 ID
    E = torch.stack([
      table(category_ids[:, f])
      for f, table in enumerate(self.tables)
    ], dim=1)              # [B, F, d]
    sum_square = E.sum(dim=1).square()
    square_sum = E.square().sum(dim=1)
    score = 0.5 * (sum_square - square_sum).sum(dim=-1)
    return score           # [B]，这里只展示二阶分支
```

lookup 不是不可训练的外部步骤。它等价于 one-hot 与 embedding 参数矩阵相乘；autograd 会将梯度传回本 batch 使用到的行。上面是形状示意，本文未将它作为完整训练工程运行验证。

8. DeepFM 与 Wide & Deep：两条不同的组合路线

8.1 DeepFM 不是枚举三阶组合

我的疑问是：“100 个 field，FM 有 4,950 个内积，那 DeepFM 是不是还要算 $\binom{100}{3}$ 个三元组合？”

不需要。DeepFM 使用两个共享 embeddings 的分支：

$$z = \text{concat}(e_1, \dots, e_F, \text{dense inputs}),$$

$$h_1 = \sigma(W_1 z + b_1), \quad h_2 = \sigma(W_2 h_1 + b_2), \quad s_{\text{deep}} = w_o^\top h_2 + b_o,$$

$$s = s_{\text{FM}} + s_{\text{deep}}, \qquad p = \text{sigmoid}(s).$$

这是示意结构，隐藏层数量不是固定两层。非线性隐藏层加最终线性输出层时，“几层”必须说明计数口径。

FM 分支直接求和二阶交互；deep 分支接收原始 embeddings 的 concat，通过 MLP 隐式学习复杂组合，而不是先把 FM 的 4,950 个内积再送进 MLP。

MLP 有能力表达涉及三、四个乃至更多输入的关系，但不意味着每种高阶交互都会被充分、可靠地学到，也没有 DCN 那样清晰的显式最高多项式阶数。

8.2 “输出概率是中间表示，还是最终结果？”

经典 CTR 模型中，FM 和 deep 分支输出的是标量 logit 贡献，相加后 sigmoid 得到最终点击概率。

可以把 deep 分支改成输出 embedding，接到更大的网络；也可以把其他特征加入输入。这样的扩展合理，但要与原版 DeepFM 的定义分开。

8.3 Wide & Deep：选交叉形式，系数由数据学

我想到：既然某些具体规则重要，为什么不在最终分数里加一个规则项?

Wide & Deep 的 wide 分支可以写成：

$$s_{\text{wide}} = b + \sum_j a_j \phi_j(x),$$

其中 $\phi_j(x)$ 可以是原始特征，也可以是预定义的交叉指示特征。例如某个地域与某类主题同时出现时为 1，否则为 0。

$$s = s_{\text{wide}} + s_{\text{deep}}, \qquad p = \text{sigmoid}(s).$$

人为决定的是“提供哪些交叉特征”，a_j 则可以通过同一个 loss 联合学习。它与上线后人为固定加分系数的业务规则不是一回事。

FM 用 embedding 内积参数化交互；wide 分支可以为某些具体组合保留独立参数。两者表达与共享方式不同。

9. DCN v1：亲手展开一次显式多项式交叉

9.1 x0 到底多大?

设 100 个 field，每个 embedding 16 维，则：

$$x_0 = \text{concat}(e_1, \dots, e_{100}) \in \mathbb{R}^{1600}.$$

这里已经是 embeddings 的 concat，**不是原始 one-hot**。不需要先压回 16 维。

x_l 是第 l 个 cross layer 的状态，仍为 1,600 维；下标 l 表示层，不表示 field。

9.2 我想自己从 x0 推 x1

起点是构造外积：

$$x_0 x_l^\top \in \mathbb{R}^{D\times D}.$$

再乘一个 $D \times 1$ 的可学习向量 w_l，压回 D 维，接 residual：

$$x_{l+1} = x_l + x_0(x_l^\top w_l) + b_l.$$

利用结合律，不必真正创建 $D \times D$ 外积：先算标量 $x_l^\top w_l$，再乘 x_0。每层 cross 参数与计算均为 $O(D)$ 量级。

9.3 “不就是 x0 整体缩放吗？”

忽略 bias，令：

$$a_l = x_0^\top w_l.$$

第一层：

$$x_1 = x_0 + x_0(x_0^\top w_0) = x_0(1 + a_0).$$

继续：

$$\begin{aligned} x_2 &= x_1 + x_0(x_1^\top w_1) \\ &= x_0(1 + a_0) + x_0(1 + a_0)a_1 \\ &= x_0(1 + a_0)(1 + a_1) \\ &= x_0 + x_0a_0 + x_0a_1 + x_0a_0a_1. \end{aligned}$$

我的关键追问： “看起来还是 x0，只是整体保留度不同，哪里来的二阶三阶？”

答案必须展开到坐标：

$$(x_0a_0)_i = \sum_j (w_0)_j (x_0)_i (x_0)_j,$$

这是二阶项；而

$$(x_0a_0a_1)_i = \sum_{j,k} (w_0)_j (w_1)_k (x_0)_i (x_0)_j (x_0)_k,$$

就是三阶项。

所以两个事实同时成立：

1. 无 bias 时，状态方向仍与 x_0 共线，表达存在明显约束。
2. 缩放系数本身依赖输入，因此输出是输入的非线性多项式，产生了更高阶交互。

不能因为写成 $x_0 \times$ scale，就把 scale 当作与输入无关的常数。

9.4 深度、残差与阶数

堆叠 L 个 cross layer，显式多项式最高次数为 $L+1$。因此两个 cross layer 的 cross 分支最高到三次项。

这是坐标乘积的次数，不必然意味着三个不同 field；同 field 的不同维度、重复坐标也可能出现。

residual 保留低阶路径，但不保证层数越深越好。交叉阶数增加、参数估计、梯度稳定性、样本规模和正则化都影响结果。若后面还有非线性 MLP，也不能再把完整模型说成“最高只到三次”。

10. DCNv2：从整体缩放走到逐维交叉

10.1 对 v1 最自然的追问

既然 v1 只有一个标量门控，能否让 1,600 个坐标分别得到不同的混合结果?

用完整矩阵替换向量：

$$g_l = W_l x_l, \qquad W_l \in \mathbb{R}^{D\times D},$$

$$x_{l+1} = x_l + x_0 \odot (W_l x_l + b_l).$$

上式采用 DCNv2 常见的 bias 置于逐元素乘积内部的写法；与前面 v1 外加 bias 的位置不要混淆。

我确认形状时的说法是：“x_l 过一个 1,600×1,600 矩阵，生成 1,600 个数，再跟 x_0 elementwise 相乘？”——对。

更准确叫交叉调制值，不要求是概率、正数或归一化权重。

坐标形式：

$$(x_{l+1})_i = (x_l)_i + (x_0)_i \left(\sum_j W_{l,ij}(x_l)_j + b_{l,i} \right).$$

v1 中所有输出坐标共享同一套标量聚合；v2 可以对不同输出坐标学习不同组合。

10.2 是不是数据越大，v2 > v1 > DeepFM > FM?

不能建立这样的固定排序。参数规模、优化、特征、交互复杂度、稀疏度、正则化与实现都影响结果。

“某论文中 v2 更好”是特定设置的证据，不是“数据量到某个阈值就必然胜出”的定律。对工业级数据，更需要看计算预算、公平调参、实际指标和多次实验。

11. 低秩分解与 LoRA：压缩的究竟是什么

11.1 DCNv2 的 W 可以分解

完整 $D \times D$ 矩阵每层需要 D^2 参数。令：

$$W_l = U_l V_l^\top, \qquad U_l, V_l \in \mathbb{R}^{D\times r},$$

则：

$$W_l x_l = U_l (V_l^\top x_l).$$

参数变为 $2Dr$，但整个 W 的 rank 被限制在 r 以内。

11.2 “LoRA 也是这样吗？不对，它是 delta W？”

这个修正很关键：

场景	参数化	哪部分受低秩限制
原始权重低秩训练	$W = BA$	整个 W
LoRA	$W = W_0 + \frac{\alpha}{r} BA$	更新量 ΔW

其中 $B \in \mathbb{R}^{m\times r}$、$A \in \mathbb{R}^{r\times n}$，A、B 直接参与训练。LoRA 一般冻结已有 W_0。

BA 是对更新量的低秩参数化，不要求先把完整 ΔW 算出来，再做一次 SVD。

从零训练当然也可以把整个 W 参数化成 BA，但这会限制完整层的容量，不能把 LoRA rank=8/16 的经验直接照搬。

11.3 LLM pretraining 的实验旁支

讨论顺带查到 Spectron 的原生低秩预训练实验：按输入维度比例设 rank，在其 94M 模型的消融中，比例 0.25 和 0.4 的 perplexity 接近，而 0.125 明显退化。见参考文献 [9] 附录 B.2。

这提供了具体实验起点，但不是通用工业经验；优化方法、模型规模和训练预算都必须一起看。梯度低秩投影、低秩增量累积为高秩权重，也不能直接当成“整个 W 始终低秩”的同一件事。

本篇不继续展开这条旁支，回到推荐架构。

12. DLRM：保留交互分数，再学习非线性组合

12.1 从 sparse 与 dense 两类输入开始

我提出年龄可以分桶，user ID 可以 hash 后查 embedding；这当然可以。

另一种处理是：dense 数值先按需要归一化或变换，避免量纲差异，再过 bottom MLP：

$$h_{\text{dense}} = \text{MLP}_{\text{bottom}}(x_{\text{dense}}) \in \mathbb{R}^d.$$

归一化不一定非要映射到 0–1，也可以标准化、log 变换等。关键是数据分布、语义和稳定性。

sparse fields 查表得到 $e_1, \dots, e_F \in \mathbb{R}^d$。让 dense 分支也输出 d 维，就可以参与相同的内积交互。

12.2 我说：“后面可以接 FM / DeepFM / DCN 了”

作为架构设计，确实可以选择这些交互方式。但经典 DLRM 的具体路线是：

$$\mathcal{E} = [h_{\text{dense}}, e_1, \dots, e_F],$$

$$p_{fg} = \langle \mathcal{E}_f, \mathcal{E}_g \rangle, \qquad f < g,$$

$$z = \text{concat}\left(h_{\text{dense}}, [p_{fg}]_{f<g}\right),$$

$$s = \text{MLP}_{\text{top}}(z), \qquad p = \text{sigmoid}(s).$$

如果 F=100 个 sparse fields，加一个 dense 表示后共有 101 个向量，所以有 $\binom{101}{2} = 5,050$ 个内积分数。别把这个数与仅 100 个 sparse fields 的 4,950 混在一起。

12.3 “DeepFM 也是这样吧？”

不是同样的 MLP 输入：

架构	MLP 看见什么	二阶交互怎么处理
FM	无必需的 deep 分支	内积后直接求和
加权 FM 扩展	可没有 MLP	内积后学线性加权
DeepFM	原始 embeddings 的 concat	FM 求和分支与 deep 标量相加
DLRM	各对内积分数 + dense 表示	top MLP 学交互分数的非线性组合

12.4 这样建模的好处与代价

把内积交互显式算出来，top MLP 不必从头构造同样的乘法关系，还能学多个匹配分数之间的非线性条件。

例如某两种匹配同时高时加分，只有其中一种高时不加分。这比“所有内积直接相加”更灵活。

但每个向量对被压成一个标量，会丢信息。它不保证比 concat 全部原始 embeddings 更强。

另外，“更紧凑”也不是无条件成立。DeepFM deep 输入规模约为 Fd；DLRM 交互数为 $O(F^2)$。F 很大、d 很小时，保留全部 pair 可能反而更长。需要代入真实维度比较。

13. DIN：让候选决定哪些历史行为更重要

13.1 这还是用户行为建模吗?

对，起点还是用户历史 action list，每条历史有一个 embedding e_i。

新问题是：同一段历史，面对不同候选，重要行为可能不同。

- 候选是喜剧，喜剧相关历史可能更重要。

- 候选是惊悚，惊悚相关历史可能更重要。

因此需要：

$$h_u(v) = \text{Aggregate}(e_1, \dots, e_T \mid v).$$

与独立生成一个固定用户向量不同，这里的用户兴趣表示依赖当前候选。

13.2 推演第一步：“内积呀”

我先想到用内积衡量相关性：

$$s_i = e_i^\top v,$$

然后把分数当权重乘回历史向量并求和：

$$h_u(v) = \sum_i s_i e_i.$$

这是合理的起始结构。不过内积可能为负、尺度也可能变化，是否加激活、归一化或其他稳定化处理，需要明确定义；不能自动把它当成概率。

13.3 推演第二步：“concat 后 MLP 算一个值”

为了更灵活地学候选与历史关系：

$$s_i = \text{MLP}_{\text{activation}}([e_i, v]), \qquad h_u(v) = \sum_i s_i e_i.$$

这个相关性 MLP 对所有历史位置共享参数。它从最终监督目标中学习“哪类历史对哪类候选更相关”。

这解释了 DIN 的核心：local activation unit 生成与候选相关的历史权重。原文还设计了显式交互输入、Dice 和 mini-batch aware regularization；上面是帮助推导的简化表达，不是完整逐层复现。

原论文局部激活并不要求像标准 attention 那样做 softmax 归一化，因此不要将“attention”自动等同为“权重和必须为 1”。

13.4 推演第三步：“聚合后当然和候选 v 一起预测”

$$s = \text{MLP}_{\text{rank}}([h_u(v), v, \text{other features}]),$$

$$p(\text{click}) = \text{sigmoid}(s).$$

采用点击 BCE 后，梯度可回传到预测 MLP、相关性 MLP 与历史/候选 embeddings，联合训练。

13.5 它和之前 self-attention 双塔有什么不同?

维度	候选无关的历史 encoder	DIN 式候选相关聚合
历史关系	可用 self-attention 建模历史之间关系	根据候选判断历史相关性
输出	一个候选无关用户状态	每个候选对应一个兴趣表示
标准 ANN 双塔	可直接使用	不能把候选相关聚合直接作为统一用户查询向量
常见位置	retrieval / ranking 用户特征	ranking

这两者也可以组合：历史先经过序列 encoder，再针对候选聚合。但那是扩展架构，不能把所有 attention 模型都称为 DIN。

14. 今天涉及的 Loss：BCE、in-batch CE、logQ 与多正例

本节收拢已经讨论过的基础，不提前展开 Part 2 的完整 loss 谱系。

14.1 点击预测：BCE

对于一条曝光样本，label $y_i \in \{0, 1\}$，模型输出 logit s_i：

$$p_i = \text{sigmoid}(s_i),$$

$$\ell_i = -y_i \log p_i - (1 - y_i) \log(1 - p_i).$$

若样本权重为 $\omega_i \geq 0$，可定义：

$$L = \frac{\sum_i \omega_i \ell_i}{\sum_i \omega_i}.$$

它表达的是逐样本点击概率目标。FM、DeepFM、Wide & Deep、DCN、DLRM、DIN 都可以配合这样的监督；架构与 loss 不是一一绑定。

实际实现宜直接使用稳定的 logits 版本 BCE。若采用负采样，模型概率对真实曝光分布的校准还需要考虑采样机制。

14.2 双塔：in-batch negatives

一个 batch 有 B 条正交互 (u_i, v_i)，计算：

$$S_{ij} = \frac{q_{u_i}^\top k_{v_j}}{\tau}.$$

最简单的单正例目标：

$$\ell_i = -\log \frac{\exp(S_{ii})}{\sum_{j=1}^{B} \exp(S_{ij})}.$$

B×B 个 user-item 配对中，有 B 个对角正例，B(B−1) 个非对角候选负配对。

这里“batch 内其他视频是负例”只是训练构造，不代表日志证明用户不喜欢它们。它们更准确是采样负例或未标注配对，存在 false negatives。

14.3 我强调：随机 batch 不应该为了干净而篡改真实数据

随机分 batch 时，同一个用户、多条正例、重复视频都可能出现。不能为讲解方便就强行假设它们不存在。

但“保留原始数据”与“正确构造 loss 的正负关系”不矛盾。重复 item 若被当成另一行的负例，可能制造相互矛盾的监督；需要对 item ID、已知正例集合和 loss mask 有明确定义。

对于 user i 的已知正例集合 P_i，一种目标是：

$$\ell_i = -\log \frac{\sum_{j\in P_i} \exp(S_{ij})}{\sum_{j\in A_i} \exp(S_{ij})},$$

其中 A_i 是本行有效候选集合。它鼓励把概率质量放在正例集合中。

另一种是分别监督每个正例：

$$\ell_i = -\frac{1}{|P_i|} \sum_{p\in P_i} \log \frac{\exp(S_{ip})}{\sum_{j\in A_i} \exp(S_{ij})}.$$

两者不是等价写法：后者更明确要求每个正例都得到概率，前者可能主要依靠其中一个强正例。到底选哪一种，取决于任务语义，不能只说“多正例 loss”而不写公式。

14.4 logQ：纠正采样分布，不是自动消除热门偏置

若视频 j 在候选采样中被选到的概率为 $q(v_j)$，一种 sampled-softmax 近似会使用：

$$\widetilde{S}_{ij} = S_{ij} - \log q(v_j).$$

实际公式应对应具体采样方式、是否有放回、重复去重和正例处理；不能只凭“logQ”这个名字无条件套用。

核心直觉是：热门 item 更频繁作为 batch 候选出现，采样分布会影响训练，校正项让模型考虑这种差异。

它并不能自动消除曝光偏差、位置偏差或推荐系统反馈回路。若正样本几乎都来自热门内容，仅有 logQ 不会创造未观察到的用户长尾偏好。

后续可以研究真实曝光未点击、位置、观看质量、IPS 等，但不能把“未点击”等同为确定负偏好，也不能直接凭位置靠后就任意增权。

14.5 样本表与训练/推理边界

一条训练记录可以包含：

user_id / video_id / session_id / scenario
prediction_time / label_time
user_features_as_of_prediction_time
video_features_as_of_prediction_time
history_before_prediction_time
label / sample_weight / sampling_metadata

batch ID 通常由 DataLoader/采样过程生成，不必作为永久特征存入主表。

PyTorch 模型继承 nn.Module，实现 forward；loss 可以独立定义。训练通过 autograd 和 optimizer 更新参数，推理使用 eval() 以及 no_grad()/inference_mode()。**eval() 本身不等于关闭梯度。**

15. 附录：显著性、AUROC/NDCG 与特征重要性

15.1 指标提高，第一问不是立刻讲线上线下差异

先问：**离线提升是否可信、幅度是否有实际价值？** 然后再讨论为何线上不对应。

但也不需要把 p<0.05 当成所有模型决策的唯一门槛：效果大小、置信区间、实验功效、业务成本和预先定义的决策规则都应考虑。

15.2 p-value、alpha 与 Z

零假设：

$$H_0 : \mu_A = \mu_B.$$

若均值差除以标准误近似标准正态：

$$Z = \frac{\bar{X}_B - \bar{X}_A}{SE(\bar{X}_B - \bar{X}_A)}.$$

双侧 $\alpha = 0.05$ 的门槛为 $|Z| > 1.96$；预先指定方向的单侧检验门槛为 $Z > 1.645$。

p-value 是 H0 成立时，出现当前这么极端或更极端统计量的概率，不是 H0 为真的概率，也不是“拒绝 H0 的概率”。

alpha 是预先指定的第一类错误控制水平。p<alpha 时按对应检验规则拒绝 H0。

15.3 CTR、NDCG 与 AUROC 的区别

指标	单位贡献与聚合
CTR	若每个独立单位只有一次机会，是 0/1 的均值；多曝光用户场景则需考虑比率与聚类
NDCG	通常先对每个 query 算一个值，再对 query 聚合

指标	单位贡献与聚合
AUROC	正负样本之间的相对排序统计量，单条样本没有一个独立 AUC

AUROC 可写成：

$$\widehat{\mathrm{AUC}} = \frac{1}{n_+ n_-} \sum_{i:y_i=1} \sum_{j:y_j=0} \left[\mathbf{1}(s_i > s_j) + \tfrac{1}{2}\mathbf{1}(s_i = s_j) \right].$$

正负 pair 共享样本，不能把所有 pair 当成互相独立的实验单位。

对于同一测试集上的两个模型，可使用配对 bootstrap：每轮抽同一批测试单位，两个模型分别计算 AUC，再得到 ΔAUC。重复后估计差值的不确定性。存在 user 内相关时，考虑按 user 整组抽样。

也可以使用 DeLong 方法估计 AUC 差值的方差，不是 AUROC 必须依赖 bootstrap。

15.4 “我真有 1,000 个独立测试集合也能算吧？”

可以，对每组计算配对 AUC 差，再分析这些差值。但这估计的是**按该分组协议得到的集合级 AUC 差的平均**。

各子集 AUC 的平均一般不等于全量 pooled AUC，因为 pooled AUC 还包含跨子集的正负配对。分组大小、正负比例和来源不能被忽略。

固定测试集上的 bootstrap，是在对来自总体的有限样本不确定性做推断；它不会替代多训练 seed 对训练随机性的评估。

15.5 Permutation test 与 permutation importance

Permutation test： 在零假设和可交换性/实验随机分配规则下，置换标签或分配，构造统计量的零分布，估计 p-value。

它和 Z 检验的目标相似，区别主要在于如何得到零分布。不是因为实验单位少才“发明”它，也不会增加原始数据的信息量。

大流量 A/B test 的随机波动通常更小，但不是被忽略了：标准误或随机化推断都在考虑这种不确定性。

Permutation importance： 模型固定，打乱某一特征，衡量预测指标下降，回答模型依赖该特征多少。

对于 embedding，要把整条向量作为一个特征组在样本之间重排：

原样本 A: [其他特征 A, embedding A, label A]
打乱以后: [其他特征 A, embedding B, label A]

不逐维独立打乱，否则破坏了向量内部结构。重复置换可以看稳定性。

但若 embedding 与 user ID 等特征高度相关，重排会制造不自然的组合；重要性也可能受特征替代影响。它衡量的是固定模型对输入的依赖，不等于因果价值，也不等于删去该特征重新训练后的损失。

GBDT 还常用分裂次数和 gain importance；DNN 可用梯度归因等。不同方法回答的问题与偏差不完全相同。

16. Part 2 从哪里继续

今天模型主线已经覆盖：

模型	希望学什么	今天最关键的理解
Two-tower	用户与视频的匹配空间	两塔独立编码，训练与服务都要守边界
历史 self-attention encoder	行为之间的上下文与序列关系	最后状态可作为用户表示，注意时间截断
FM	类别之间的两两交互	用 embedding 内积替代独立类别 pair 系数
DeepFM	显式低阶 + 隐式复杂关系	FM 与原始 embedding MLP 并行，共享 embeddings
Wide & Deep	具体组合记忆与表示泛化	提供交叉特征，系数可联合学习
DCN v1	有阶数边界的显式坐标交叉	展开 scale，就能看到二阶、三阶项
DCNv2	更灵活的逐维交叉	标量门控变成矩阵混合，可低秩参数化
DLRM	交互分数之间的非线性关系	保留每对内积，再送 top MLP
DIN	候选相关的用户兴趣	候选决定历史权重，主要用于 ranking

下一篇先把模型收尾，再进入 loss：

1. **DIEN**：历史行为不等于潜在兴趣；如何建模兴趣演化，辅助监督为什么有用。
2. **SASRec 对照**：把已经推过的 causal history encoder 对齐到序列推荐模型，重点补训练目标，不重学 Transformer。
3. **多兴趣召回 / MIND**：如何让同一个用户输出多个向量，如何分配行为、训练和检索。
4. **Loss 主线**：从 pointwise 到 pairwise/listwise；BCE 与 softmax CE 分别优化什么；RankNet/BPR、margin/hinge、温度、hard negative、多正例与辅助 loss。已熟悉的只做对齐。

每一步继续按照同一套问题推进：

> 样本是什么？希望学到什么关系？最自然的公式是什么？哪里不够？如何修改结构或 loss？梯度和服务约束能否成立？

参考文献与阅读顺序

本文的教学顺序不等于论文发表顺序。下列链接用于核对原始架构；正文中的动机推演是理解方式，不代表作者发表时的完整研究过程。

1. Steffen Rendle. **Factorization Machines**. ICDM 2010. 官方项目与论文目录，论文 DOI。
2. Cheng et al. **Wide & Deep Learning for Recommender Systems**. 2016. 论文。
3. Guo et al. **DeepFM: A Factorization-Machine based Neural Network for CTR Prediction**. 2017. 论文。
4. Wang et al. **Deep & Cross Network for Ad Click Predictions**. 2017. 论文。

5. Wang et al. **DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems**. 预印本。
6. Naumov et al. **Deep Learning Recommendation Model for Personalization and Recommendation Systems**. 2019. 论文。
7. Zhou et al. **Deep Interest Network for Click-Through Rate Prediction**. KDD 2018，预印本始于 2017. 论文。
8. Hu et al. **LoRA: Low-Rank Adaptation of Large Language Models**. 论文。
9. Janson et al. **Stabilizing Native Low-Rank LLM Pretraining**. ICML 2026. 论文。

Part 2 预备阅读：

- Zhou et al. **Deep Interest Evolution Network for Click-Through Rate Prediction**. AAAI 2019. 论文。
- Kang and McAuley. **Self-Attentive Sequential Recommendation**. ICDM 2018. 论文。
- Li et al. **Multi-Interest Network with Dynamic Routing for Recommendation at Tmall**. CIKM 2019. 论文。

本篇停在这里：模型不是名字的集合，而是对数据中某种关系的结构化表达。先自己把关系推出来，再看论文如何实现、如何验证，阅读会清楚得多。